The Journal of Open Source Software

GraphCalc: A Python Package for Computing Graph Invariants in Automated Conjecturing Systems

Randy Davila [1,2]

1 Department of Computational Applied Mathematics & Operations Research, Rice University, United States **2** RelationalAI, United States

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Christoph Junghans

Reviewers:

- @szhorvat
- @aadinoyiibrahim

Submitted: 30 May 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

`GraphCalc` is a Python library for computing a wide range of graph-theoretic invariants using a blend of exact enumeration, solver-based optimization, and integrations with `NetworkX`. In addition to supporting `NetworkX` graphs, the package includes internal data structures for graphs and polytopes, facilitating experimentation and extension.

Originally developed to support **automated conjecturing systems**, `GraphCalc` powers the *GraffitiAI* family of programs (Brimkov et al., 2024; Davila, 2025a, 2025b), where it is used to generate and organize numerical and Boolean data on simple graphs and polytopes in order to formulate computer-generated conjectures.

`GraphCalc` includes tools for computing both classical and specialized invariants. These include well-known parameters such as the **independence number**, **clique number**, and **chromatic number** (each computed exactly using optimization models), as well as lesser-known but actively studied invariants such as:

- Spectral properties related to **graph energy** (Li et al., 2012),
- Degree-sequence-based invariants like **residue** (Favaron et al., 1991), **annihilation number** (Levit & Mandrescu, 2022), and **Slater number** (Gentner & Rautenbach, 2017),
- Dynamic coloring-based parameters such as the **zero forcing number** and its many variants (AIM Minimum Rank – Special Graphs Work Group, 2008; Amos et al., 2015; Davila et al., 2018; Davila & Henning, 2019),
- The widely studied **domination number** and its generalizations (**total**, **connected**, **Roman**, **k-domination**, etc.) (Haynes et al., 2021, 2023; Henning & Yeo, 2010),
- Structural constraints defined by forbidden subgraphs, such as being **claw-free**, **triangle-free**, **diamond-free**, and **bull-free**.

Altogether, `GraphCalc` implements over 100 graph-related functions, including invariant computation, spectral analysis, structural testing, and graph generation. Many of these are unavailable in any other Python package, and all are computed exactly using integer programming or specialized algorithms. The library is also well-suited for rigorous graph-theoretic research, enabling users to verify counterexamples and explore structural hypotheses.

Statement of Need

Many aspects of research in graph theory, as well as in automated reasoning and conjecture generation on discrete structures rely on the accurate computation of graph invariants across large collections of graphs. While important and general purpose libraries such as `NetworkX` (Hagberg et al., 2008) and `igraph` (Csárdi & Nepusz, 2006) offer essential tools for graph manipulation and visualization, they provide limited support for computing many of the

specialized and computationally difficult invariants central to a large amount of graph (and polyhedral) theoretical research.

`GraphCalc` fills this gap by offering a unified, solver enhanced framework for computing both classical and advanced graph invariants. Many parameters are evaluated via linear-integer programming models, relying on the `PuLP` optimization toolkit (Mitchell et al., 2011) and the COIN-OR CBC solver (Forrest & Lougee-Heimer, 2005) to compute exact solutions for many NP-hard invariants, or fallback to exhaustive enumeration when necessary. The library currently supports over 100 functions across a wide range of categories, including:

- Classical invariants: **independence number**, **clique number**, **chromatic number**, **vertex cover**, and **matching number**;
- Over a dozen **domination-type invariants**, including **total**, **Roman**, **double Roman**, **restrained**, **outer-connected**, and **k-domination**;
- Specialized parameters like **zero forcing**, **positive semidefinite forcing**, **connected forcing**, and **k-power domination**;
- Degree-based and spectral invariants such as **residue**, **annihilation number**, **Slater number**, **spectral radius**, and **algebraic connectivity**;
- Structural graph properties and Boolean predicates for detecting trees, regularity, planarity, forbidden subgraphs, and more.

Unlike broad symbolic systems such as SageMath (The Sage Developers, 2020), `GraphCalc` is purpose-built for seamless integration into Python **automated conjecturing systems**. It serves as the computational engine for the *TxGraffiti* and *Optimist* systems (Davila, 2025a, 2025b), which depend on accurate invariant computations to discover, test, and rank symbolic conjectures that relate seemingly unrelated structural properties of graphs and polytopes.

Features

`GraphCalc` provides a comprehensive suite of graph-theoretic computations, combining familiar interfaces with powerful solver-based backends. It supports both `NetworkX` graph objects and its own lightweight `SimpleGraph` and polytope graph types, allowing users to flexibly model, analyze, and visualize a wide variety of graph structures. Key features include:

- **Extensive invariant support**: Compute classical parameters such as the **chromatic number**, **maximum clique**, **vertex cover**, **matching number**, and **independence number**, alongside specialized invariants like **residue**, **Slater number**, and **annihilation number**.
- **Domination and zero forcing variants**: Includes exact computation of more than a dozen domination-type invariants (e.g., **total**, **Roman**, **double Roman**, **restrained**) and advanced forcing parameters like **positive semidefinite zero forcing** and **k-power domination**.
- **Spectral and structural analysis**: Supports computation of eigenvalues (adjacency and Laplacian), **spectral radius**, **algebraic connectivity**, and detection of structural properties (e.g., planarity, claw-freeness, subcubicity).
- **Graph and polytope generators**: Includes generators for classical graphs (e.g., **Petersen**, **Barabási–Albert**, **Watts–Strogatz**) and convex polytopes (e.g., **cubes**, **tetrahedra**, **fullerenes**).
- **Batch evaluation**: Compute multiple invariants across many graphs using `compute_graph_properties_dataframe`, returning results in a clean, tabular format suitable for analysis and experimentation.
- **Visualization and usability**: Native support for drawing graphs and polytopes; functions are type-annotated and tested, with complete documentation hosted online.

Example Usage

```python
import graphcalc as gc
from graphcalc.generators import petersen_graph
```

```python
# Create the Petersen graph
G = petersen_graph()

# Compute basic invariants
print("Independence number:", gc.independence_number(G))    # Output: 4
print("Chromatic number:", gc.chromatic_number(G))          # Output: 3
print("Connected?", gc.connected(G))
```

You can also analyze polytope graphs and batch-compute properties:

```python
from graphcalc.polytopes.generators import cube_graph, octahedron_graph
graphs = [cube_graph(), octahedron_graph()]
functions = ["order", "size", "spectral_radius", "independence_number"]

df = gc.compute_graph_properties_dataframe(functions, graphs)
print(df)
```

Relevance to Automated Discovery

Automated mathematical discovery has deep roots, dating back to early symbolic logic systems in the 1950s with Wang's *Program II* (Wang, 1960), and gaining momentum in the 1980s with systems like Fajtlowicz's *Graffiti* (Fajtlowicz, 1988) and DeLaViña's *Graffiti.pc* (DeLaViña, 2005). These programs demonstrated that computers could do more than verify mathematics—they could help discover it, particularly through the generation of conjectures based on structural graph invariants. Collectively, such systems have led to well over 100 published mathematical conjectures, many appearing in top-tier journals.

`GraphCalc` continues in this tradition, serving as the computational foundation for modern systems such as *TxGraffiti* and *Optimist* (Brimkov et al., 2024; Davila, 2025b), which generate symbolic conjectures by analyzing numerical patterns across families of graphs and polytopes. To support this process, `GraphCalc` supplies high-quality, reproducible invariant data—ranging from classical quantities like **chromatic number** and **independence number** to more specialized parameters such as **positive semidefinite zero forcing**, **annihilation number**, and **Slater number**.

While `GraphCalc` is not designed for massive-scale network analysis, it excels in the domain where most mathematical conjectures are actually formed: small to medium-sized graphs that can be drawn, visualized, and reasoned about—graphs that fit comfortably within the scope of human mathematical intuition. This design philosophy echoes that of Fajtlowicz, who emphasized working with "small but interesting" graphs as the foundation for conjecture generation. By transforming these graphs into rich numerical profiles, `GraphCalc` enables automated systems to detect symbolic patterns and formulate conjectures that are not only structurally meaningful but also aligned with the way mathematicians discover and explore new ideas.

Acknowledgements

The authors gratefully acknowledge David Amos and Boris Brimkov for their foundational support during the development of `GraphCalc`. David Amos provided technical insight and early design feedback, while Boris Brimkov's mathematical input helped guide the selection and implementation of key graph invariants.

DRAFT

References

AIM Minimum Rank – Special Graphs Work Group. (2008). Zero forcing sets and the minimum rank of graphs. *Linear Algebra and Its Applications*, *428*(7), 1628–1648. https://doi.org/10.1016/j.laa.2007.10.009

Amos, D., Caro, Y., Davila, R., & Pepper, R. (2015). Upper bounds on the k-forcing number of a graph. *Discrete Applied Mathematics*, *181*, 1–10. https://doi.org/10.1016/j.dam.2014.08.029

Brimkov, B., Davila, R., Schuerger, H., & Young, M. (2024). On a conjecture of TxGraffiti: Relating zero forcing and vertex covers in graphs. *Discrete Appl. Math.*, *359*, 290–302. https://doi.org/10.1016/j.dam.2024.08.006

Csárdi, G., & Nepusz, T. (2006). The igraph software package for complex network research. *InterJournal Complex Systems*, *1695*. https://doi.org/10.5281/zenodo.3630268

Davila, R. (2025a). *Automated conjecturing in mathematics with TxGraffiti*. https://doi.org/10.48550/arXiv.2409.19379

Davila, R. (2025b). *The optimist: Towards fully automated graph theory research*. https://doi.org/10.48550/arXiv.2411.09158

Davila, R., & Henning, M. A. (2019). On the total forcing number of a graph. *Discrete Applied Mathematics*, *257*, 115–127. https://doi.org/10.1016/j.dam.2018.09.001

Davila, R., Henning, M. A., Magnant, C., & Pepper, R. (2018). Bounds on the connected forcing number of a graph. *Graphs and Combinatorics*, *34*(6), 1159–1174. https://doi.org/10.1007/s00373-018-1957-x

DeLaViña, E. (2005). *Graffiti.pc*: A Variant of Graffiti. In *Graphs and discovery* (Vol. 69, pp. 71–88). American Mathematical Society. https://doi.org/10.1090/dimacs/069/05

Fajtlowicz, S. (1988). On conjectures of Graffiti. *Discrete Mathematics*, *72*(1), 113–118. https://doi.org/10.1016/0012-365X(88)90199-9

Favaron, O., Mahéo, M., & Saclé, J.-F. (1991). On the residue of a graph. *J. Graph Theory*, *15*, 39–64. https://doi.org/10.1002/jgt.3190150107

Forrest, J., & Lougee-Heimer, R. (2005). CBC user guide. In *Emerging theory, methods, and applications* (pp. 257–277). INFORMS. https://doi.org/10.1287/educ.1053.0020

Gentner, M., & Rautenbach, D. (2017). Some comments on the slater number. *Discrete Math.*, *340*(7), 1497–1502. https://doi.org/10.1016/j.disc.2017.02.009

Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. In G. Varoquaux, T. Vaught, & J. Millman (Eds.), *Proceedings of the 7th python in science conference (SciPy2008)* (pp. 11–15). https://doi.org/10.25080/TCWV9851

Haynes, T. W., Hedetniemi, S. T., & Henning, M. A. (2021). *Topics in domination in graphs*. Springer. https://doi.org/10.1007/978-3-030-51117-3

Haynes, T. W., Hedetniemi, S. T., & Henning, M. A. (2023). *Domination in graphs: Core concepts*. Springer. https://doi.org/10.1007/978-3-031-09496-5

Henning, M. A., & Yeo, A. (2010). *Total domination in graphs*. Springer. https://doi.org/10.1007/978-1-4614-6525-6

Levit, V. E., & Mandrescu, E. (2022). Some more updates on an annihilation number conjecture: Pros and cons. *Graphs and Combinatorics*, *38*(5). https://doi.org/10.1007/s00373-022-02534-7

DRAFT

Li, X., Shi, Y., & Gutman, I. (2012). *Graph Energy* (1st ed., p. 268). Springer. https://doi.org/10.1007/978-1-4614-4220-2

Mitchell, S., Dunning, I., & O'Sullivan, M. (2011). PuLP: A linear programming toolkit for python. *Optimization Online*. https://optimization-online.org/2011/09/3178/

The Sage Developers. (2020). *SageMath, the Sage Mathematics Software System (Version 9.1)*. https://doi.org/10.5281/zenodo.4066866

Wang, H. (1960). Toward mechanical mathematics. *IBM Journal of Research and Development*, *4*(1), 2–22. https://doi.org/10.1147/rd.41.0002

DRAFT